UNIVERSITE DE FIANARANTSOA

ECOLE NORMALE SUPERIEURE

SCIENCE ET TECHNOLOGIE

MATHEMATIQUES

DEVELOPPER LA PENSEE INFORMATIQUE CHEZ LES COLLEGIENS A TRAVERS SCRATCH

Encadré par : Monsieur Stefana TSILEFA

Présenté par : RAJAONARIVELO Ellonah Karel

TABLE DES MATIERES

REMERCIEMENT 5
RESUME 6
INTRODUCTION 7
Chapitre 1 : SCRATCH ET LA PENSEE INFORMATIQUE 8
1.1 La pensée informatique 8
1.1.1 Définition 8
1.1.2 Principe de la pensée informatique 8
1.1.3 Objectifs de l'enseignement de la pensée informatique au collège 8
1.2 Présentation de Scratch comme outil pédagogique 9
1.2.1 Historique et présentation de Scratch 9
1.2.2 Objectifs du Scratch 10
1.2.3. Mode d'utilisation 11
1.2.4 Exemples 14
1.3 Avantages de Scratch dans le développement de la pensée informatique 15
1.3.1. Objectifs pédagogiques qu'on peut considérer dans l'utilisation de Scratch 15
1.3.2 Hypothèses 16
Chapitre 2 : METHODOLOGIE ET EXPERIMENTATION 19
2.1. Présentation de l'échantillon 19
2.1.1 Niveaux d'Exercices 19
2.1.2 Objectif de l'Expérimentation 19
2.2 Déroulement de l'Expérimentation 19
2.3. Illustration des exercices traités par les échantillons : 20
Chapitre 3 : RESULTAT ET DISCUSSION 28
3.1 Analyse a priori des compétences des échantillons 28
3.2 Discussion des résultats 28
3.3 Compétences développées chez les échantillons et limite de Scratch 35
CONCLUSION 37
BIBLIOGRAPHIE 38

LISTE DES FIGURES

Figure 1 : Interface graphique de Scratch 11
Figure 2 : Zone de Scène 11
Figure 3 : Palette de Blocs 12
Figure 4 : Activité 1, calcul de somme de 10 premiers nombres 14
Figure 5 : Acticité 2 : devinette de nombre secret 14
Figure 6 : Activité 3 : Calcul d'aire d'un rectangle 15
Figure 7 : Exemple de traçage d'un carré 16
Figure 8 :Calcul et affichage de A 21
Figure 9 :Calcul et affichage de D 21
Figure 10:Calcule et affichage de A' et B' (étant A et B dans cet affichage) 21
Figure 11 :Calcule et affichage de D' 22
Figure 12 : Calcule et affichage de C 22
Figure 13 : Calcul et affichage de E 22
Figure 14 :Calcule et affichage de F 23
Figure 15 : Calcul et affichage de K 23
Figure 16 : Traçage du cercle et calcul du périmètre 24
Figure 17 : Calcule et affichage d'aire du cercle 24
Figure 18 : Jeu de devinette 25
Figure 19 : Calcul et affichage de A et B 26
Figure 20 : Calcul et affichage de D, F et K 26
Figure 21 : Affichage de traçage d'un carré 27
Figure 22 : Jeu de devinette 2 27

RENSEIGNEMENT

Nom : RAJAOANARIVELO

Prénoms : Ellonah Karel

Niveau : M1/MT/P

N° CE : 3722066

REMERCIEMENT

Je tiens à exprimer ma plus profonde gratitude à Dieu, qui a été ma source d'inspiration et de force tout au long de cette recherche sur ma mémoire, que la gloire Lui soit rendue, car sans Lui, rien de tout de cela n'aurait été possible.

Je souhaite aussi remercier chaleureusement mon encadreur Monsieur Stefana TSILEFA, pour son soutien précieux et ses conseils avisés. Votre expertise et votre disponibilité ont été une grande aide dans l'avancement de mon travail.

A ma famille et mes amis proche, je vous suis infiniment reconnaissant pour votre amour, votre encouragement et votre soutien inconditionnel.

Je tiens également à remercier les participants à l'expérimentation, Rayan et Feno, ainsi que leur famille. Leur participation et leur engagement dans cette expérience ont été essentiels à la réussite de cette recherche.

A tous, un grand merci pour votre soutien.

RESUME

Madagascar inclus progressivement l'informatique, plus précisément la programmation informatique dans le cursus scolaire, mais fréquemment c'est au lycée qu'on l'introduit. En se référant sur les objectifs majeurs de la programmation, on comprend que celle-ci peut permettre au développement de la pensée informatique. Différents outils sont disponibles pour enseigner cette pensée notamment les langages, les jeux, les activités débranchés (sans utiliser des outils numériques, par exemple des exercices d'algorithme sur papier). On a opté d'utiliser le langage de programmation comme outil pour vérifier si c'est l'outil le plus adapté pour le développement de la pensée information.

Pour le vérifier, on a mené l'expérience auprès de deux élèves de 6e, novices en programmation pour mesurer les compétences qui définies la pensée informatique, ainsi que sur leur motivation et leur attitude vis-à-vis des sciences, en utilisant le langage de programmation Scratch.

On s'est centré sur l'évaluation de la compétence en résolution de problème, en algorithmique en logique et sur la motivation lors de cette expérience. Notre résultat montre un effet positif sur l'utilisation de Scratch, on a pu observer un haut niveau de motivation d'un élève créatif et de nature scientifique qu'un élève littéraire.

Mots clés : pensée informatique, programmation, Scratch, collégiens

INTRODUCTION

Dans un monde de plus en plus numérique, la pensée informatique émerge comme une compétence essentielle à développer dès le plus jeune âge. A Madagascar, l'initiative d'inclure des programmes en informatique à l'école est en phase d'essor. L'utilisation d'outils numériques comme Scratch, un langage de programmation visuel et à but éducatif, offre une opportunité unique d'initier les élèves à la logique algorithmique et à la résolution de base, qui sont des principales compétences constituant la pensée informatique. Ce projet de recherche se concentre sur l'impact de Scratch sur le développement de la pensée informatique chez les élèves secondaires, précisément les élèves de 6e.

La problématique centrale de cette recherche est de comprendre comment l'utilisation de Scratch peut favoriser le développement de la pensée informatique chez les collégiens Malgaches. Plus précisément, nous nous interrogeons sur les compétences que les élèves peuvent développer grâce à cet outil et sur les défis qu'ils rencontrent dans leur apprentissage.

Les questions de recherche qui guideront cette étude sont les suivantes :

*Quelles compétences en pensée informatique peuvent être développées par les élèves à travers Scratch ?

*Quels sont les avantages et limites de cet outil dans le contexte éducatifs malgache ?

*Comment les élèves réagissent ils face à l'apprentissage de la programmation avec Scratch ?

Nous formulons l'hypothèse selon laquelle l'utilisation de Scratch comme outil pédagogique peut significativement améliorer les compétences en algorithmique, en logique et en résolution de problèmes tout en stimulant leur créativité et leur motivation.

Le plan de cette mémoire se décline en trois chapitres. Le premier chapitre est consacré à la pensée informatique et à Scratch. Nous y définirons la pensée informatique, ses principes et ses objectifs d'enseignement au secondaire, puis nous allons présenter l'historique de Scratch et son mode d'emploi et nous examinerons également les raisons pour lesquelles Scratch est un choix pertinent pour développer la pensée informatique ainsi que ces avantages autour de la programmation et de la multidisplinarité. Le deuxième chapitre portera l'expérimentation. Nous y présenterons les échantillons de notre étude et illustrerons les travaux effectués par les élèves participants. Enfin, le troisième chapitre sera dédié aux résultats et aux discussions. Nous analyserons a priori les compétences des échantillons, discuterons les résultats obtenus et mettrons en lumière les compétences développées, ainsi que les limites de Scratch.

Chapitre 1 : SCRATCH ET LA PENSEE INFORMATIQUE

1.1 La pensée informatique

1.1.1 Définition

La pensée informatique est l'ensemble de compétences et de processus cognitifs qui permettent de résoudre des problèmes de manière systématique et efficace en utilisant les concepts issus de l'informatique. Elle englobe des compétences telles que la décomposition de problèmes complexes en sous-problèmes plus simples, la reconnaissance de modèles, l'abstraction et la formulation d'algorithmes. En d'autres terme, la pensée informatique consiste à penser de manière logique et structurée pour concevoir des solutions à problèmes variés, qu'il soit lié à l'informatique ou d'autres domaines.

1.1.2 Principe de la pensée informatique

Les principes de la pensée informatique se reposent sur plusieurs concepts clés :

1. La décomposition : cela consiste à diviser un problème complexe en sous- problème plus simples. Par exemple, pour créer un jeu vidéo, un élève peut décomposer le projet en différentes tâches, telles que la conception des personnages, la création des niveaux, puis la programmation des règles du jeu

2. Reconnaissance de modèle : ce principe implique d'identifier des similitudes ou des motifs dans les données ou les problèmes. Cela aide à anticiper des solutions basées sur des expériences passées.

3. Abstraction : cela consiste à se concentrer sur les aspects essentiels d'un problème tout en ignorant les détails superflus. Cela permet de simplifier la complexité d'un problème et se concentrer sur les éléments clés nécessaires à la solution

4. Algorithmes : un algorithme est une série d'instructions ou de règles à suivre, structurés de manière logique, pour résoudre un problème. La pensée informatique encourage les élèves à concevoir des algorithmes pour structurer leur approche de la résolution de problèmes.

1.1.3 Objectifs de l'enseignement de la pensée informatique au collège

L'enseignement de la pensée informatique à Madagascar vise plusieurs objectifs tels que

*le développement des compétences en résolution de problèmes

*la favorisation de la créativité et innovation

*la préparation des élèves à un monde numérique

* promouvoir l'interdisplinarité : la pensée informatique peut être appliquée à divers domaines, tels que les sciences, les mathématiques et même les arts

* le renforcement de l'accès à l'éducation technologique

1.2 Présentation de Scratch comme outil pédagogique

1.2.1 Historique et présentation de Scratch

Le groupe Lifelong Kindergarten du Media Lab de MIT, dirigé par Mitchel Resnick, en partenariat avec l'entreprise the Playful Invention Company (Montreal), fondée par Brian Silverman et Paula Bonta, ont développé la première version de Scratch en 2003 (Scratch 0.1)

Le logiciel Scratch est traduit en 70 langues et le site web comptait plus de 115 millions d'utilisateurs enregistrés en septembre 2023 et 138 millions de projets partagés en septembre 2023. Et il s'est hissé parmi les 20 langages de programmation les plus populaire au monde au cours de 2020 selon le classement Tiobe[1]

Plusieurs pays comme Royaume_Uni, Etats Unis, France, Estonie, Finlande, Canada, Australie, Singapour, Allemagne, et d'autres pays Africains comme Rwanda, Kenya, Togo, Afrique du Sud, Ghana, Nigeria, Cote d'Ivoire, Maroc, Sénégal et Zambie ont intégré l'enseignement de la pensée informatique dans leurs programmes d'étude, notamment à travers des outils comme scratch.

Pour les chercheurs et ingénieurs du MIT Media lab, l'idée de créer un langage de programmation aussi simple d'utilisation que les blocs sont venu suite à leur nombreuse collaboration avec Lego.

L'idée était de transposer la manière dont les enfants jouent avec des briques de Lego dans un langage de programmation pour le rendre accessible aux plus jeunes et aux néophytes. Les designs des briques permettent de comprendre simplement lesquelles sont utilisables ensemble et la manière de les associer.

[1] Scratch (langage) sur fr.m.wikipédia.org

Le langage Scratch fonctionne à partir d'instructions notées sur des blocs (colorés par exemple les variables en orange, les mouvements en bleu) que les élèves sont amenés à manipuler. L'encastrement successif de différents blocs constitue le programme. Ces blocs ont différentes couleurs qui correspondent à leurs catégories d'action : mouvement, apparence, sons, stylo, données, évènements. Le but est de pouvoir essayer d'assembler les blocs de multiples manières en les associant ou dissociant facilement.

Scratch permet ainsi à l'enseignant de diffuser sa pédagogie au moyen de l'interactivité quasi-ludique des objets manipulés par ces briques.

1.2.2 Objectifs du Scratch

D'après Professeure Jeanette Wing, comme les autres disciplines tels que l'arithmétique, l'histoire, la science de la vie sont abordés dès le primaire, la pensée informatique devrait être aussi enseigner dès le plus jeune âge.

Si on enseignait l'arithmétique au primaire, on a souvent utilisé des outils comme des bâtonnés, dés, etc…

Si on enseignait l'histoire, peut être que l'enseignant faisait un petit théâtre pour que les enfants comprennent mieux l'histoire.

Si on enseignait la géographie, l'enseignant utilisait une carte géographique. Ce qu'on veut en venir c'est que pour enseigner une nouvelle discipline aux enfants, l'enseignant doit trouver un outil plus adapté pour que les élèves puissent comprendre facilement. On voit bien que pour enseigner l'arithmétique par exemple on pourra utiliser des bâtonnés, des dés, alors pour la pensée informatique, il y a plusieurs façon (outils) de l'enseigner, par exemple à travers des jeux logiques, des langages de programmations, et de l'algorithme sur papier, etc…

Plusieurs chercheurs défendent l'idée que l'apprentissage de programmation a un impact positif sur le développement de la pensée informatique, un méta analyse de Sun et Zhou (2022) confirme cet état de fait en révélant un effet modéré en faveur d'impact positif de l'apprentissage de la programmation sur les compétences en pensée informatique.

Scratch a pour objectif l'enseignement de la programmation aux enfants et adolescents, en mettant en œuvre visuellement des concepts de base tels que les boucles, les tests, les affectations de variables. Il est dynamique tant qu'il permet de modifier le code du programme en cours d'exécution. Le code est inscrit sous forme de briques en couleurs et en plusieurs langues (70 langues), donnant la possibilité à l'enfant de choisir sa langue maternelle.

En utilisant Scratch, les enfants apprennent aussi à découper une tache en sous-taches et à résoudre de problèmes selon les principes de la pensée computationnelle ou pensée informatique.

1.2.3. Mode d'utilisation

* Interface de Scratch

Zone de script : c'est ici qu'on va glisser et déposer des blocs de code pour créer des scripts

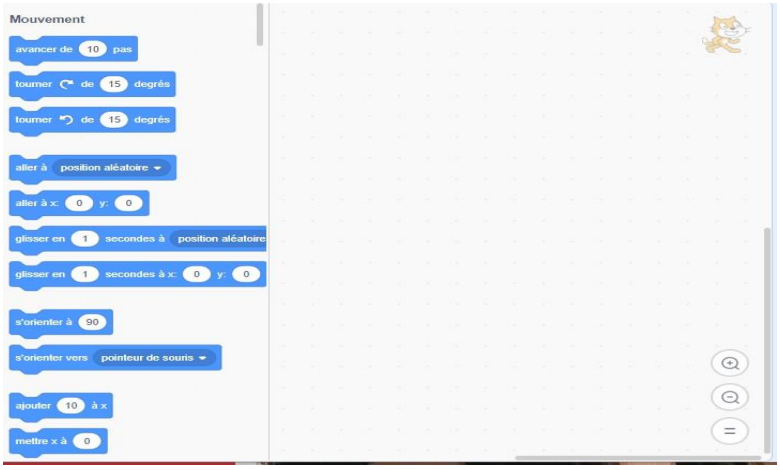


Figure 1 : Interface graphique de Scratch

Les blocs de mouvement colorés en bleu sont les blocs à glisser et à déposer dans la zone de script, celle de la droite.

Zone de scène : c'est l'endroit où le projet prend vie, il s'y trouve des personnages (sprites) et les arrières plans

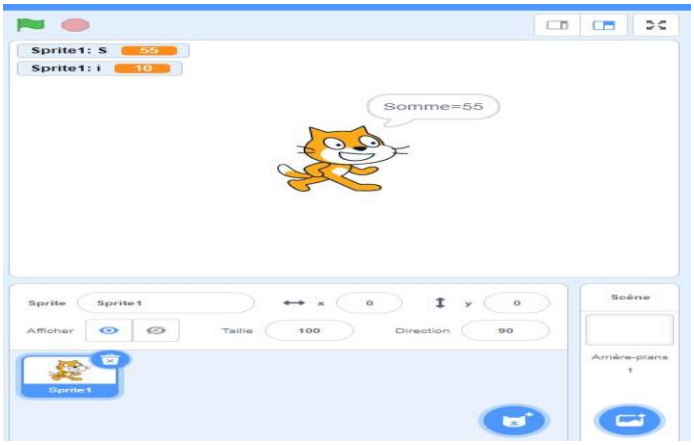


Figure 2 : Zone de Scène

Palette de blocs :il s'y trouve les différents blocs selon leurs couleurs



Figure 3 : Palette de Blocs

*Les blocs et ses utilisations

Mouvement (bleu foncé) : ce bloque permet de déplacer le sprite. On peut l'utiliser pour créer des animations

Exemple : avancer de 10 pas

Apparence : ce bloc permet d'afficher quelques choses par exemple des textes, solutions numérique dans la zone de la scène, et cela aussi permet de modifier l'arrière-plan etc.

Exemple : dire bonjour ! qui affiche 'bonjour !' dans la zone de la scène

Son : joue le son, cela peut être utilisé pour rendre un projet plus interactif

Exemple : jouer le son [miaou][2] jusqu'au bout

Evènement : ce bloc permet de lancer l'exécution du projet scratch

[2] On peut enregistrer d'autres son, mais 'miaou' est le son par défaut

Exemple : quand le drapeau vert est cliqué, quand la touche espace est pressée

Contrôle : ce bloc permet de répéter une / des instruction(s) et cela permet aussi de mettre une condition

Exemple : répéter indéfiniment, répéter [10[3]] fois, si  alors,

Capteur : ce bloc peut faire beaucoup de choses différentes, cela peut être utilisé dans le une structure conditionnelle grâce à sa certaine forme compatible à celui du bloc de contrôle dans la structure conditionnelle, ce bloc permet aussi de demander quelque chose à l'utilisateur et attendre une réponse

Exemple : touche espace pressée ?

Demander [quel est ton nom][4]

Opérateur : ce bloc contient des opérateurs arithmétiques (+, -, *, /), comparaison, opérateurs logiques (et, où) on peut aussi y manipuler des nombres aléatoires

Exemple : nombre aléatoire de 1 à 10

Variables : ce bloc permet de définir une ou plusieurs variables, il permet aussi d'affecter une/des valeurs à une/plusieurs variables, et permet aussi de créer une liste

Exemple : définir une variable, mettre [ma variable][5] à [0][6]

[3] Modifiable, 10 pas par défaut
[4] Modifiable, on peut entrer des types de donnée entiers ou réels (c'est-à-dire des nombres) ou des types de données caractères ou sous forme de texte exemple 'nom'
[5] Variable par défaut dans scratch
[6] Valeur par défaut assigner à 'ma variable'

1.2.4 Exemples

Activité 1 : Calculer la somme des 10 premiers nombres

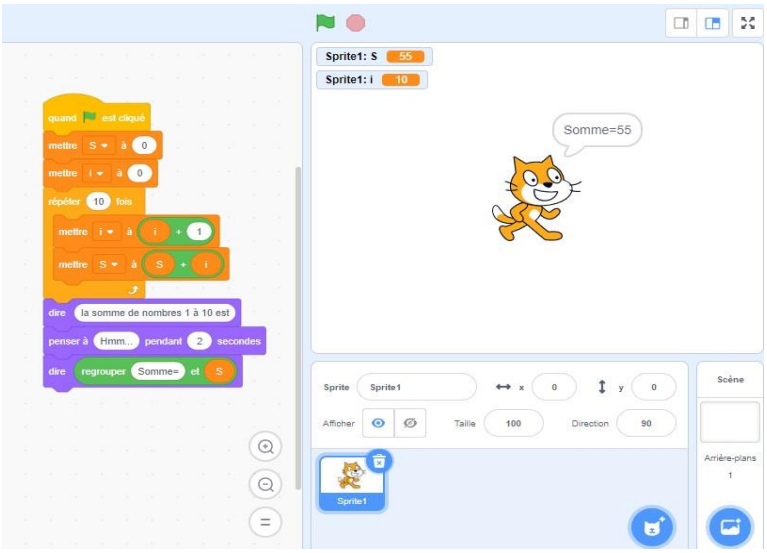


Figure 4 : Activité 1, calcul de somme de 10 premiers nombres

Activité 2 : demander à l'utilisateur de deviner le nombre secret de 1 à 20

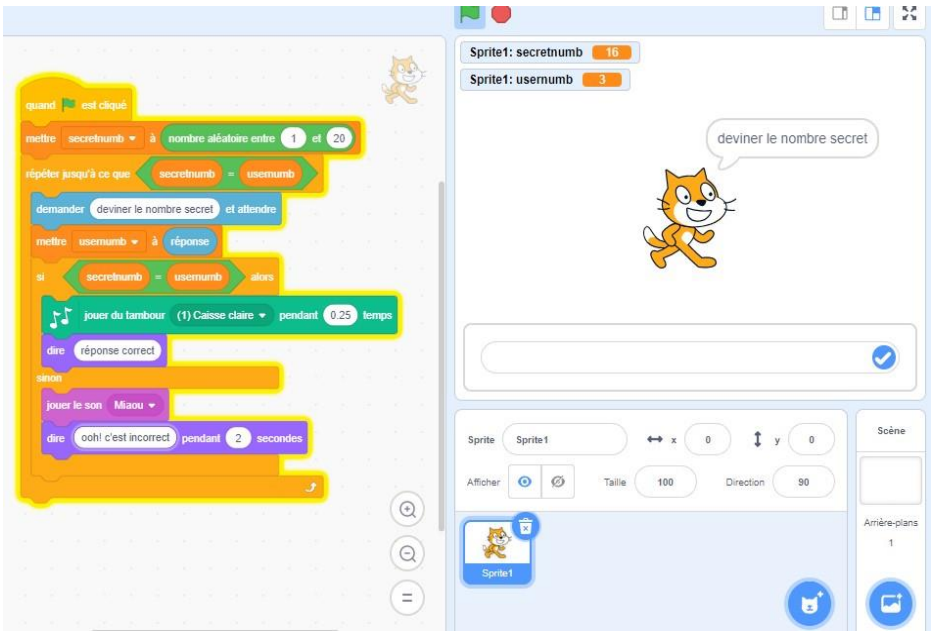


Figure 5 : Acticité 2 : devinette de nombre secret

Remarque : le sprite1 : usernumb = 0 quand on commence à exécuter le code, puis après la première compilation, usernumb de la zone de scène prend la valeur de usernumb précédent

Activité 3 : calcul d'Aire d'un rectangle

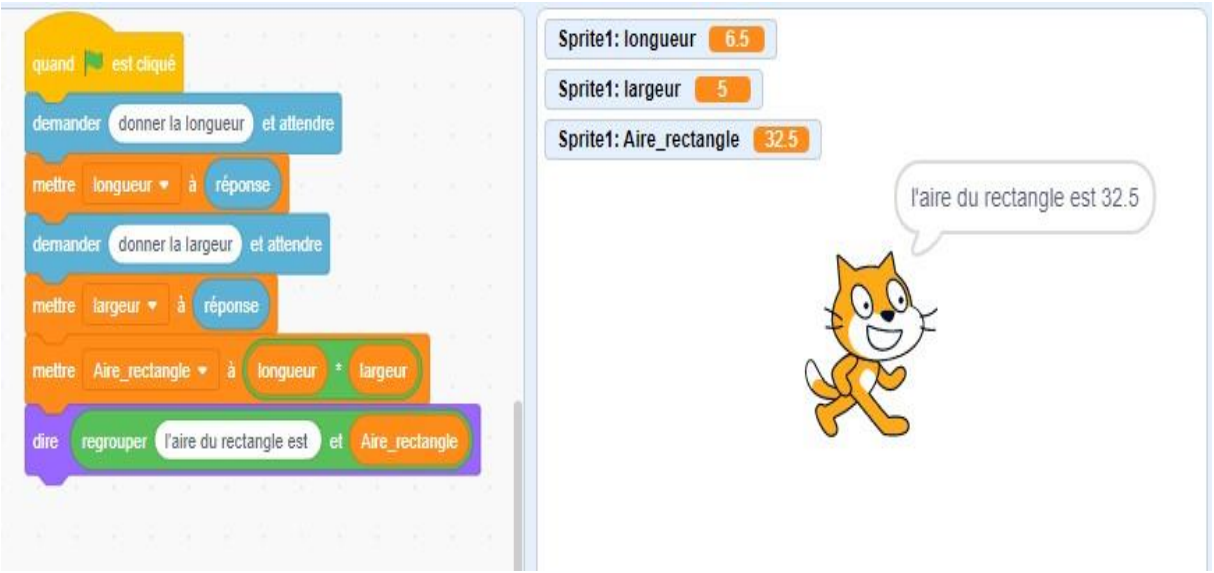


Figure 6 : Activité 3 : Calcul d'aire d'un rectangle

1.3 Avantages de Scratch dans le développement de la pensée informatique

1.3.1. Objectifs pédagogiques qu'on peut considérer dans l'utilisation de Scratch

*Autour de la programmation :

- Au niveau du syntaxe et structure : familiariser les élèves avec la syntaxe de Scratch et la structure des programmes, y compris l'utilisation des variables

-création de projet : encourager les élèves à créer des éléments de la programmation, comme des jeux ou des animations pour appliquer leurs connaissances

*autour de l'algorithmique :

-Elaboration des algorithmes : apprendre aux élèves des algorithmes simples et les notions sous-jacentes (instructions, séquence, boucles, structure conditionnelle, etc.), permettant de résoudre un problème, (il faut tenir compte des étapes des instructions du projet : c'est un peu ça l'algorithme dans scratch)

-utilisation de la logique : développer la capacité à penser logiquement et à utiliser des structures conditionnelles pour prendre des décisions dans leurs programmes

-le débogage : enseigner aux élèves de savoir s'apercevoir et à identifier leurs erreurs et à les corriger

Par exemple : si un élève exécute un programme et il ne se passe rien, il doit apercevoir qu'il y a erreur dans son code et chercher à identifier l'erreur puis le corriger

*autour de la multidisplinarité, soulignons les mathématiques

-intégration des mathématiques : utiliser Scratch pour enseigner des compétences de mathématiques de base comme l'addition, soustraction, multiplication, division, comparaison, et même Scratch aborde déjà la notion de logique (et / où) (parce que notre élèves cibles sont les collégiens et d'habitude, la notion de logique n'est pas trop souvent abordée), les graphes comme traçage de cercle, triangle, carré etc…

-application pratique : encourager les élèves à appliquer des concepts mathématiques dans des projets scratch, comme le calcul d'Aire, périmètre etc… dans l'application pratique aussi par exemple le traçage d'un carré :

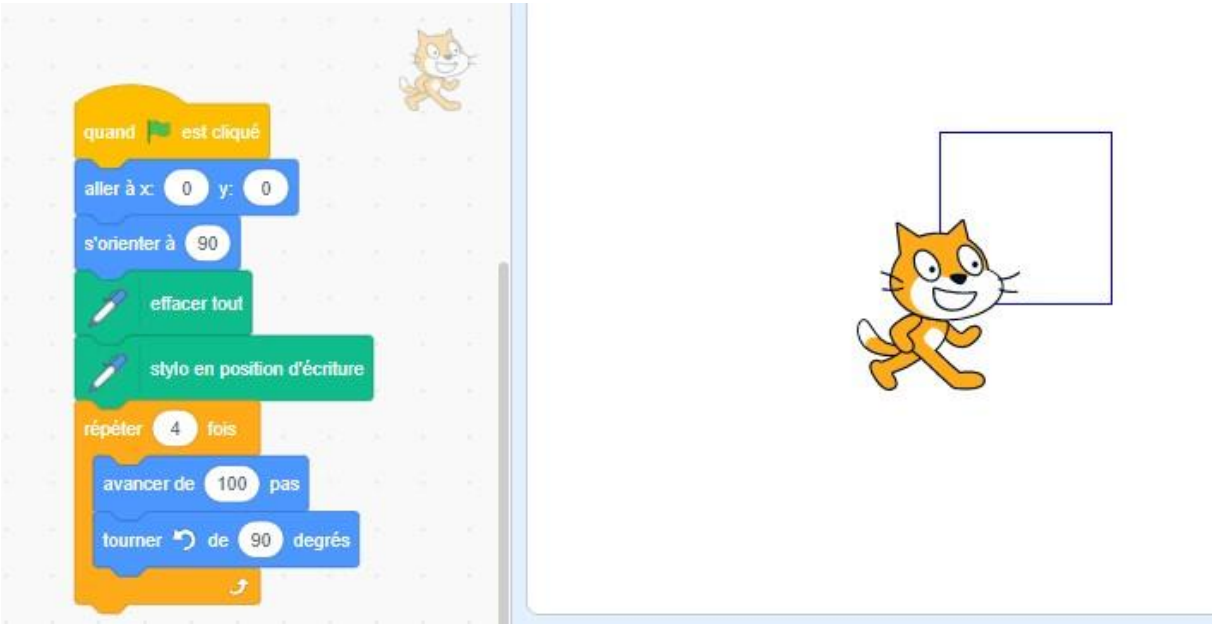


Figure 7 : Exemple de traçage d'un carré

1.3.2 Hypothèses

Avant toute chose, rappelons d'abord ce que c'est la pensée informatique pour vérifier que Scratch contribue vraiment à son développement. La pensée informatique est un processus de

résolution de problèmes qui implique de concepts et des techniques issus de l'informatique. Elle se caractérise par les compétences suivantes : savoir décomposer un problème assez complexe en sous-problème plus simples ; savoir réfléchir aux tâches à accomplir pour résoudre un problème en termes d'étapes et d'action, ce qu'on appelle algorithme ; savoir décrire les problèmes et les solutions à différents niveaux d'abstraction, ce qui permet d'identifier des similitudes entre problème et de pouvoir par suite réutiliser des éléments des solutions par exemple, tel est un problème et je veaux avoir telles solutions. Et dans un autre programme on peut réutiliser quelques solutions de l'autre projet pour résoudre un nouveau projet.

Il est clair qu'il y a beaucoup d'autres méthodes de développer la pensée informatique, mais pourquoi il est préférable de la développer à travers de la programmation. D'abord, la programmation offre une approche pratique et interactive qui engage les élèves dans le processus d'apprentissage, en écrivant du code, les élèves apprennent à décomposer des problèmes complexes en étapes plus simples. Ensuite la programmation encourage la créativité. A travers la programmation, un élève pourrait développer sa pensée algorithmique et logique qui sont le cœur de la programmation et aussi celle de la pensée informatique. Des professeurs et chercheurs comme Jeannette Wing, David Evans, Apedjinou,A. et Kouawo, C.A.A[7] défendent l'idée que la programmation contribue au développement de la pensée informatique.

D'après ces objectifs cités ci-dessus, dans cette section, on verra pourquoi il est préférable d'utiliser la programmation Scratch et pas d'autres méthodes pour développer la pensée informatique, parce que oui, il y a beaucoup d'autres méthodes comme l'utilisation de jeux et défis tels que les jeux logiques, des énigmes, etc…

Alors on verra les atouts qu'apporte la programmation Scratch dans le développement de la pensée informatique.

Tout d'abord, on pense qu'il offre une interface visuelle et intuitive qui permet aux utilisateurs, en particulier les enfants, de créer des programme en glissant-déposant des blocs de codes, de comprendre facilement les concepts de base de la programmation, d'après l'article Sticef – Vol.28 n°3 – 2021, Technologie pour l'apprentissage de l'informatique de la maternelle à l'université, cette article aligne les codes en scratch et python et on voit bien scratch pourrait être une transition douce à la programmation en python.

[7] Computational thinking : A digital Age skills for everyone ; the importance of teaching programming in Schools ; formation des élèves du primaire à la programmation informatique avec Scratch : Observation et analyse de développement des compétences sous-jacentes à la pensée.

Ensuite, Scratch encourage la créativité et l'expérimentation, en créant des projets Scratch, on veut toujours améliorer notre code, quant à l'expérimentation un utilisateur peut exécuter en temps réelle son code. Les utilisateurs peuvent aussi créer des jeux et des animations qui stimulent leurs imagination (tout en leur enseignant des concepts fondamentaux de la programmation comme les boucles, les conditions et les évènements), par exemple une création du jeu de lutin qui échappe à un obstacle, ce jeu renforce beaucoup la logique et la décomposition de problèmes et bien sur l'algorithme

De plus, on pense que scratch permet aussi d'enseigner d'autres disciplines comme les mathématiques que ce soit numérique ou géométrie, par exemple la résolution d'une équation, utilisation des opérateurs arithmétiques, calcul d'aire, périmètre, traçage des graphes tels que les triangles, cercle, carré etc. …, la langue française par exemple à l'aide d'un QCM de synonymes, antonymes, etc…

Enfin, on pense aussi que Scratch peut favoriser également la collaboration et le partage en ligne, comme son slogan l'indique 'Imagine, program, Share' ou 'Imagine, programme, partage !', il permet aussi aux utilisateurs de recevoir des commentaires, et cela peut inciter les autres membres à créer des projets Scratch et finalement cela permettra de développer la pensée informatique chez les utilisateurs.

Chapitre 2 : METHODOLOGIE ET EXPERIMENTATION

2.1. Présentation de l'échantillon

L'expérimentation a été réalisée à travers un entretien avec deux élèves Rayan, élève de 6e au collège Kolejy Maria Manjaka Talatamaty .et Feno, 6e au CEG RAKOTOZAFY Alphonse. Elle a consisté en une série d'exercices classés par niveaux de difficulté : facile, moyen et difficile.

2.1.1 Niveaux d'Exercices

**Niveau Facile : Utilisation de blocs simples tels que mouvement, apparence, événements et opérateurs.

**Niveau Moyen : Intégration des blocs du niveau facile avec des boucles, des variables et des outils de dessin.

**Niveau Difficile : Utilisation de blocs sonores, d'animations et de jeux.

2.1.2 Objectif de l'Expérimentation

L'objectif principal était d'évaluer si Scratch peut être un outil efficace pour développer la pensée informatique. Nous avons examiné les compétences clés associées à cette pensée, notamment l'algorithmique, la décomposition de problèmes en sous-problèmes, la logique et l'abstraction.

Les exercices étaient principalement axés sur les mathématiques de niveau 6e, incluant des aspects numériques (nombres entiers et rationnels) et géométriques (figures, aire et périmètre) et un jeu qui nécessite de la logique.

2.2 Déroulement de l'Expérimentation

L'expérimentation s'est déroulée en plusieurs étapes :

1. Introduction à la Pensée Informatique : On a expliqué aux échantillons ce que c'est la pensée informatique et comment elle se manifeste, une personne comment a une pensée informatique
2. Présentation de Scratch : on lui a brièvement expliqué Scratch et son mode d'emploi.
3. Exercices Manuels : on lui a fait d'abord traité les exercices manuellement avant de les réaliser sur Scratch.
4. Progression des Niveaux :

Nous avons commencé par des blocs simples tel que le bloc de l'apparence, l'opérateur, l'évènement, vérifiant son travail sur Scratch, ce qui lui a permis d'apprendre les lois d'opération et l'importance des signes dans les calculs.

Ensuite, nous avons progressé vers des exercices de niveau moyen, puis difficile, en intégrant des boucles et d'autres blocs plus complexes.

2.3. Illustration des exercices traités par les échantillons :

- Exercices traités par Rayan

D'abord, comme on l'a dit récemment, les exercices sont organisés par niveau :

Niveau facile :

Objectifs :

°compétence en logique : sur la façon d'utiliser les blocs opérateurs, sur la manière de penser quel que soit le problème

°Compréhension d'autre discipline en utilisant Scratch, ici la mathématique : règle de priorité, règle de calcul

°pensée critique : essayer de trouver l'erreur dans le programme et le corrigé

Calculer et afficher :

° A= (+3) + (-4) + 8

° D= (-1) + (+1) – 3

° A'= 2741 – 1999

° B'= 4759 + 272

° D'= (+1) – (-37) – 27

° C= 47 * 103

° E= 1273 : 12

° F= 2+3*(-5)

° K= 4 + (-3) + 5 * (-3) + [-4 + (-3)]

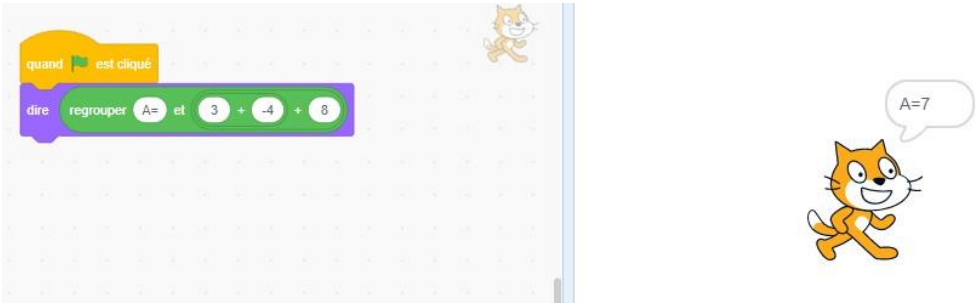


Figure 8 :Calcul et affichage de A

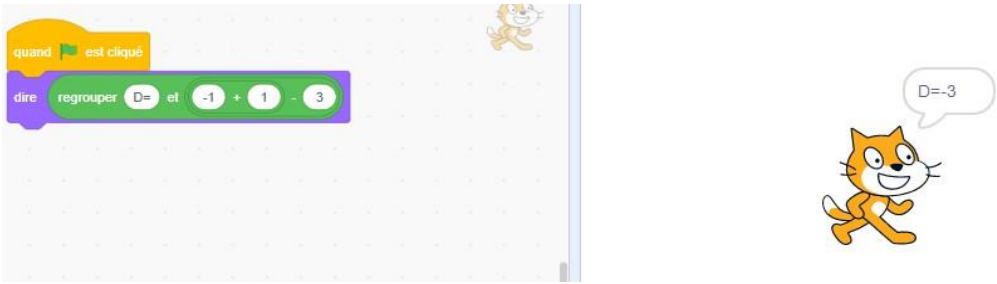


Figure 9 : Calcul et affichage de D

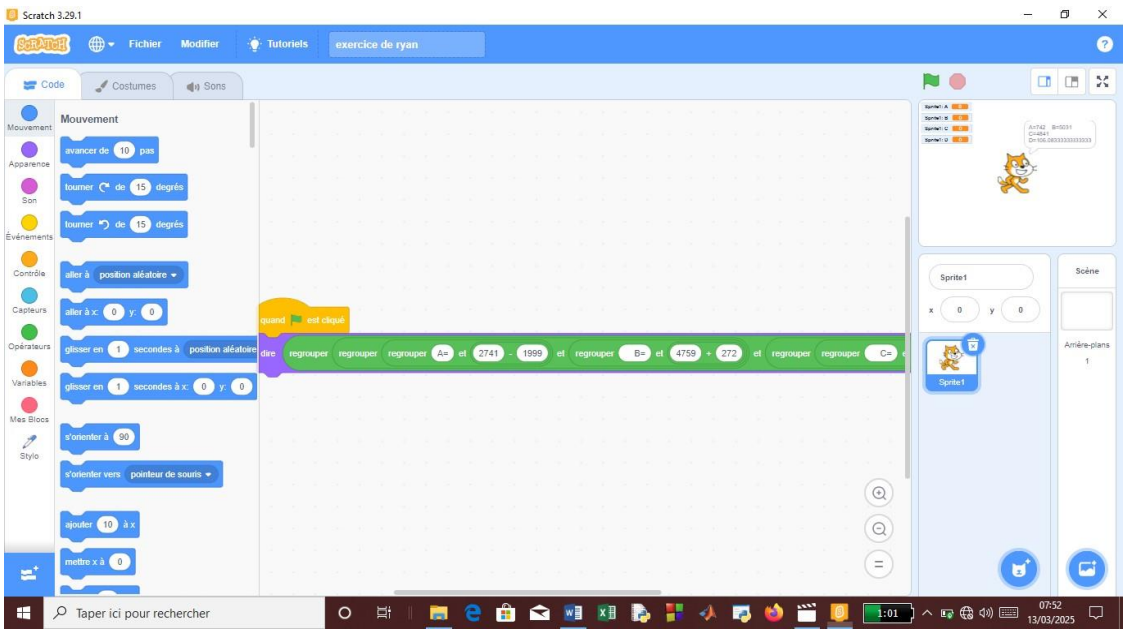


Figure 10:Calcule et affichage de A' et B' (étant A et B dans cet affichage)

Remarque : on constate ici que partant du simple calcul et simple affichage d'un calcul, Rayan a pu afficher plusieurs calculs à la fois dans son projet (A, B, C, D) dans cette image récente

L'utilisation de bloc opérateur est très importante dans ces exercices de niveau facile

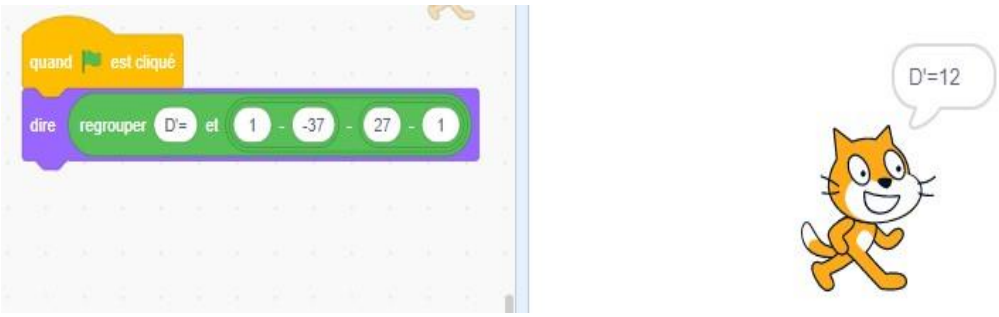


Figure 11 :Calcule et affichage de D'

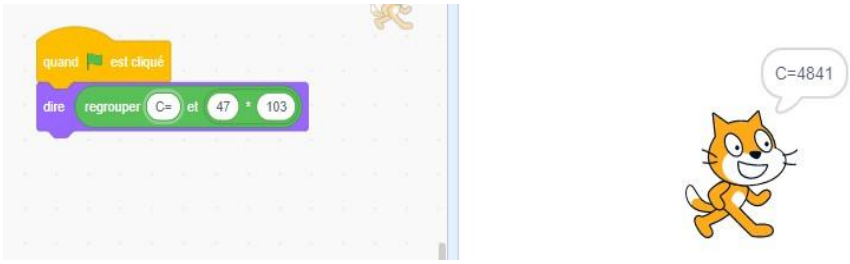


Figure 12 : Calcule et affichage de C

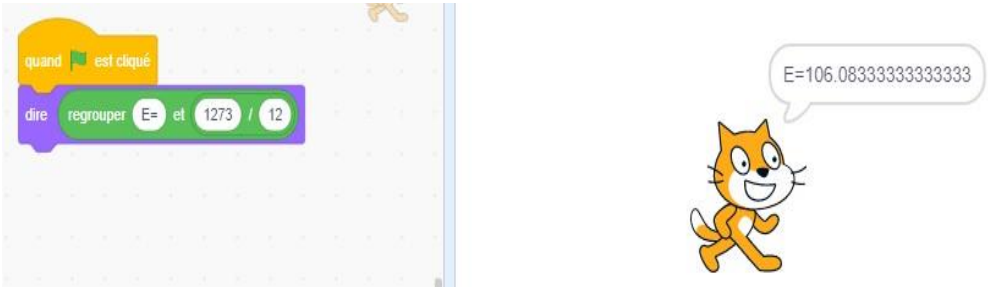


Figure 13 : Calcul et affichage de E

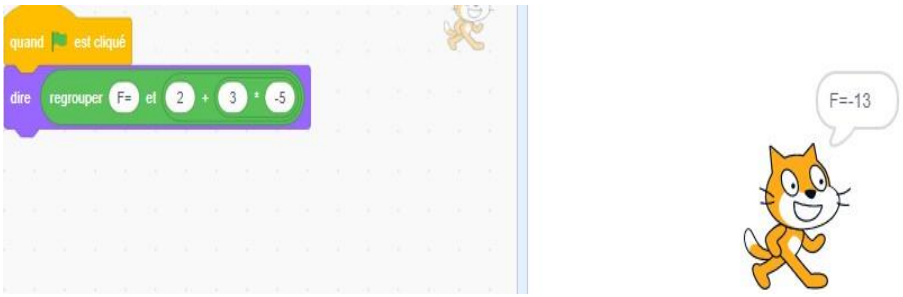


Figure 14 :Calcule et affichage de F

Niveau moyen :

Objectifs :

°Compétence en algorithme : savoir bien organiser les étapes pour résoudre un problème

°Compétence en logique

°Comprendre les concepts géométriques

°Compétence en résolution de problème : on n'a pas donné le rayon pour calculer l'aire et le périmètre, alors l'élève est invité à trouver les façons à avoir un rayon, en changeant au fur et au mesure sur le programme, mais l'idéal est de demander à l'utilisateur (bloc demander et attendre)

° Traçage et calcul du périmètre et aire d'un cercle

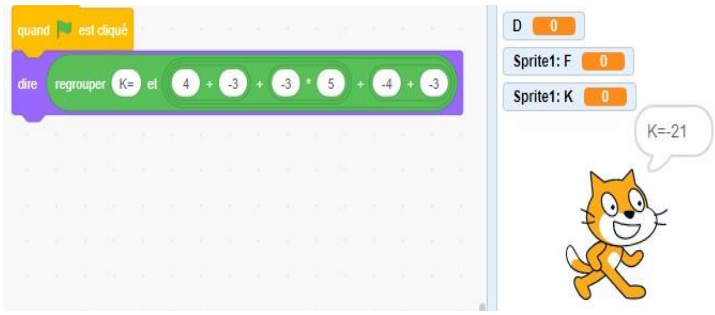


Figure 15 : Calcul et affichage de K

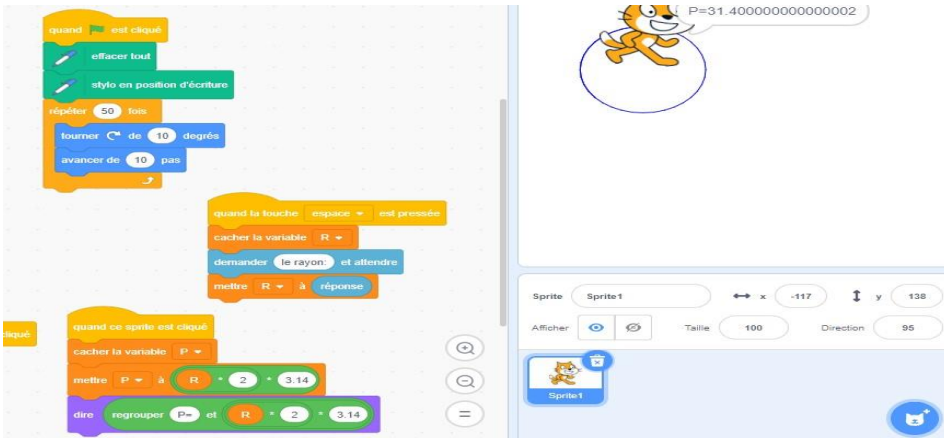


Figure 16 : Traçage du cercle et calcul du périmètre

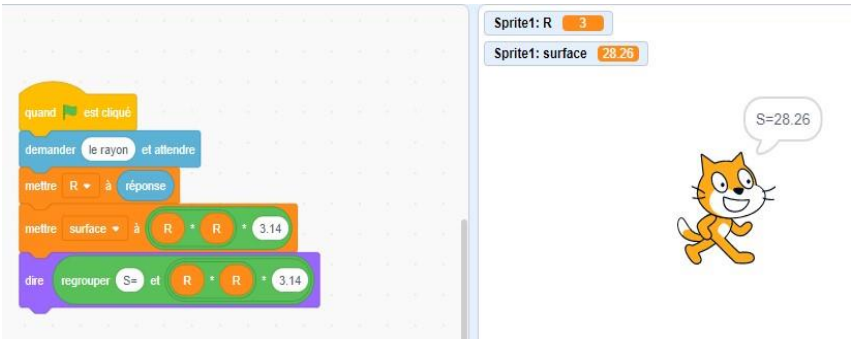


Figure 17 : Calcule et affichage d'aire du cercle

Niveau difficile :

Objectifs :

°Compétence en logique conditionnelle et en gestion des entrées utilisateurs (demander et attendre)

°Apprendre à créer des interactions dynamiques, à utiliser des nombres aléatoires des blocs

°Compétence en algorithme : bien structurer un programme de manière cohérente

°Savoir décomposer un taches complexes en sous taches plus simples

° Jeu de devinette de nombre

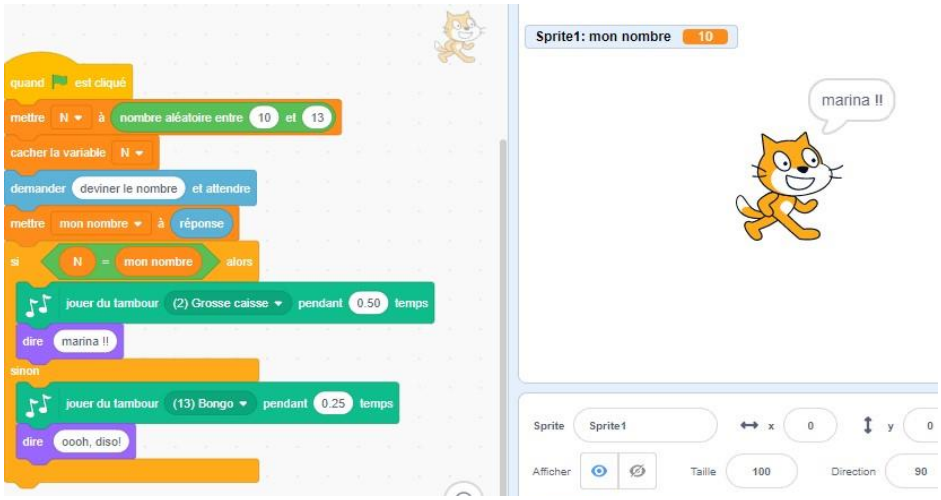


Figure 18 : Jeu de devinette

- Exercices traités par Feno :

Feno a traité les mêmes exercices que Rayan, mais il a été un peu lent que ce soit au niveau du calcul sur papier et en Scratch, donc avec lui, on n'a pas pu traiter tous les exercices traités avec Rayan.

Les objectifs de ces exercices sont les mêmes que ceux des objectifs dans les exercices traités par Rayan puisque ces exercices sont les mêmes

Niveau facile : Calculer et afficher

A= (+3) + (-4) + 8

B= 1 – (-37) – 27 – (+1)

D= (-1) + (+1) – 3

F= 2 + 3*(-5)

K= 4 + (-3) + 5*(-3) + [-4 + (-3)]

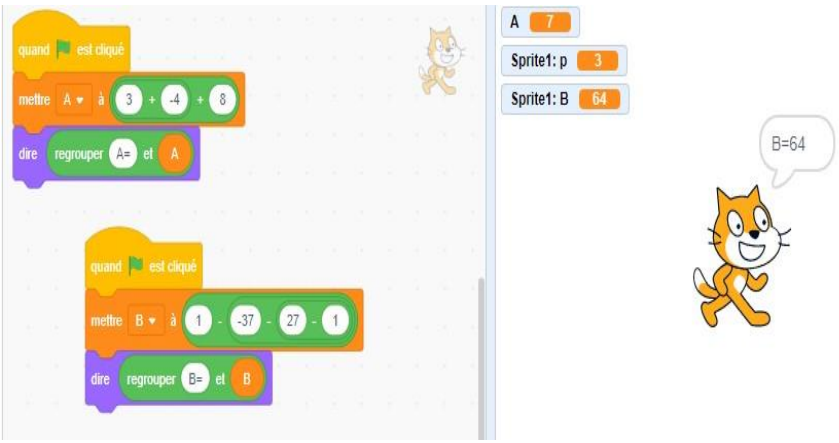


Figure 19 : Calcul et affichage de A et B

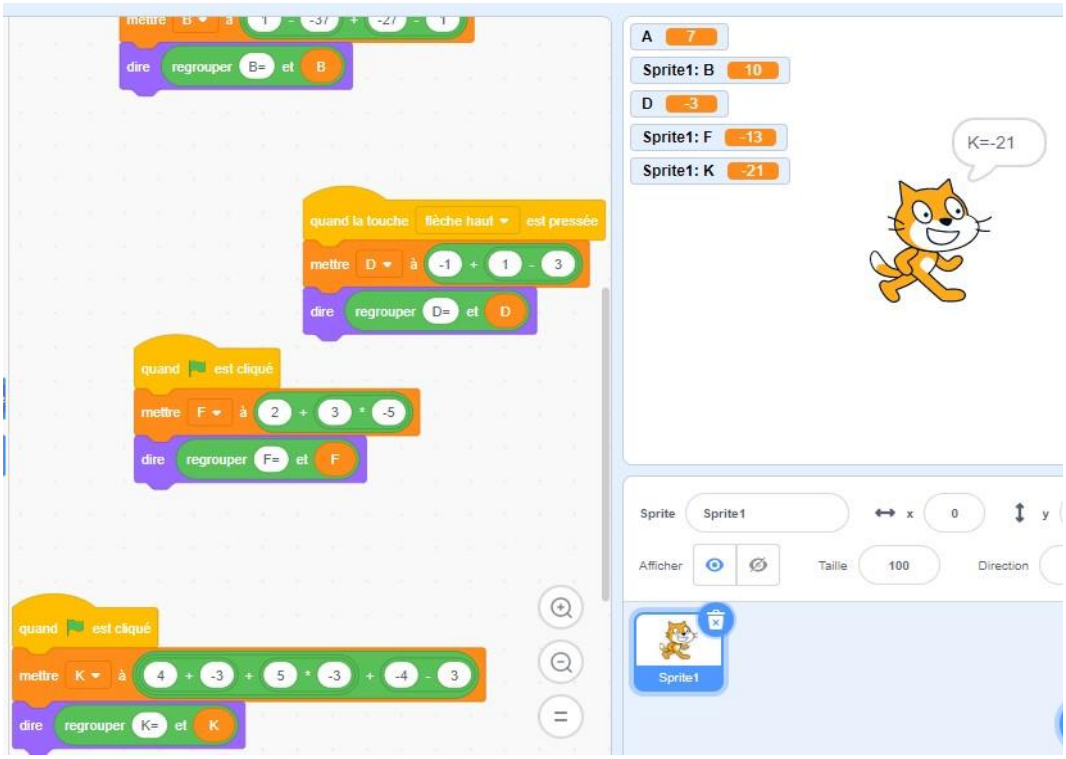


Figure 20 : Calcul et affichage de D, F et K

Niveau moyen : Traçage d'un carré

Dans cette partie, on lui a donné un indice sur les principaux blocs à utiliser, ce sont le mouvement, control

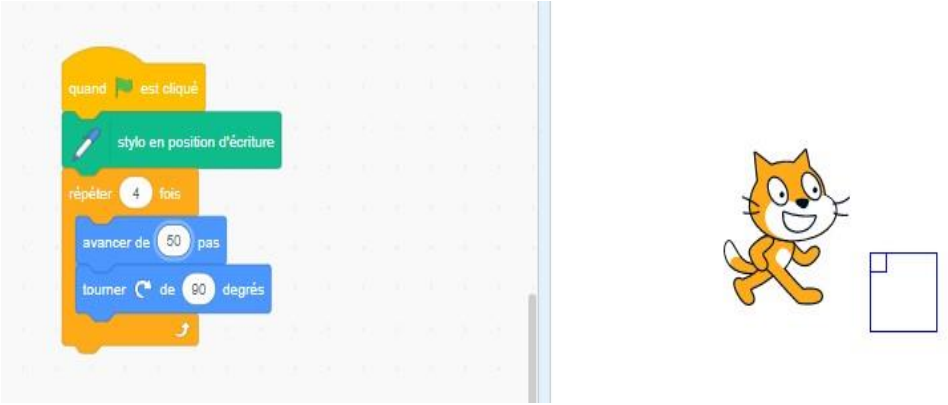


Figure 21 : Affichage de traçage d'un carré

Niveau difficile : jeu de devinette

L'utilisateur doit deviner le nombre assorti du nombre aléatoire de 1 à 10

On lui a donné un indice sur les blocs à utiliser, Feno a beaucoup besoin d'accompagnement sur cet exercice.

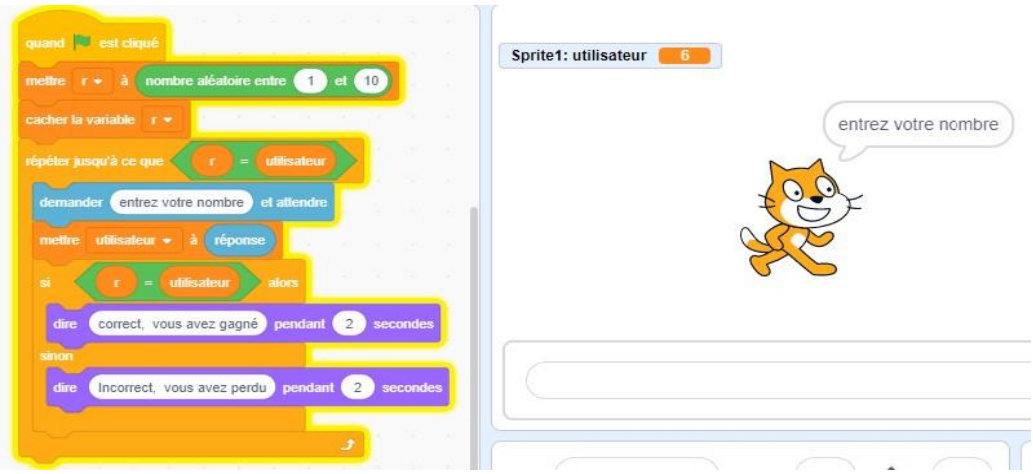


Figure 22 : Jeu de devinette 2

Lors de cette expérimentation, on a constaté que le fait que scratch organise les instructions par des blocs colorés est très utile car les enfants les souviennent facilement.

Chapitre 3 : RESULTAT ET DISCUSSION

3.1 Analyse a priori des compétences des échantillons

En général, avant notre expérimentation, Rayan avait déjà une pensée algorithmique, constaté lors de sa manière de résoudre les calculs numériques, il met une étape d'opération, par exemple en calculant A= (+3) + (-4) + 8, il met une accolade calculant d'abord (+3) + (-4) puis le résultat de ce calcul +8, i.e. (-1) + 8. Alors ça a facilité l'apprentissage de la pensée informatique.

Avant l'expérimentation, Feno. Possédait déjà aussi une pensée algorithmique, on l'a pu constater au moment de réalisation de calcul sur papier, il met des étapes de calcul pour résoudre les exercices du niveau facile.

3.2 Discussion des résultats

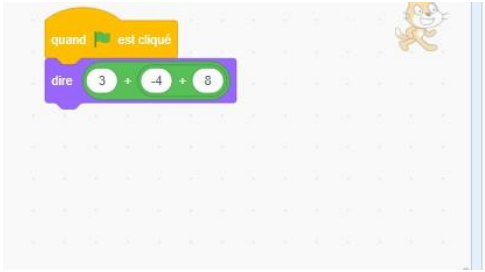
Maintenant, on va parler du résultat des exercices traités par Rayan (illustré dans le chapitre 2)

Niveau difficile :

Exercice de calcul de :

A= (+3) + (-4) + 8

On voit que le résultat de son calcul, manuellement ou sur Scratch a été correct mais la manière ou il a utilisé le bloc opérateur pour le calculer n'est pas le plus concise

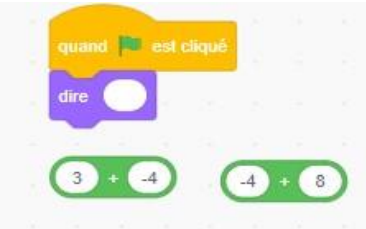


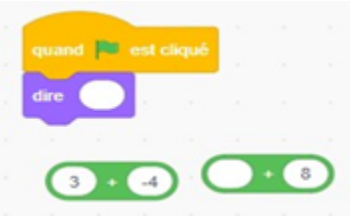


On voit que c'est vrai, mais en décomposant ce résultat, on constate qu'on peut améliorer cette réponse.

Dans sa réponse décomposée, on voit qu'il a fait (+3) + (-4) dans le bloc opérateur à gauche et (-4) +8 celui de la droite on voit la redondance de -4, mais dans la réponse corrigée, en voulant

également présenter le calcul comme sur la question, on fait d'abord (+3) + (-4) dans le bloc) gauche puis on a un espace libre avant l'opérateur + du second bloc à droite et on met 8 après cet opérateur, l'objectif c'est de mettre le premier bloc du gauche dans le vide sur le bloc de la droite. L'image ci- dessous peut mieux l'expliquer.





Dans l'exercice de calcul de B, Rayan adopté la même méthode pour faire le calcul, comme le nombre d'opération à faire a augmenté, il commencé à avoir un peu de problème lors de la résolution

Voyons dans l'image ultérieur la décomposition de son calcul

1er étape

2e étape

3eétape



D'après cette image, il a dû changé au moins deux fois les valeurs dans l'opérateur, c'est redondant. Remarquons aussi qu'il a fait un erreur sur -27, car il voulait présenter le calcul tel que le sujet, alors sa méthode n'est pas trop fiable. Alors que le résultat attendu est :



Rayan a eu un peu de difficulté sur la résolution de K= 4 +(-3) + 5*(-3) + [-4+(-3)]

Voici son résultat manuel :

Puis, il l'a vérifié sur Scratch, et c'était là que le problème d'utilisation du bloc opérateur incité ci-dessus est survenu, il utilisait sa méthode de calcul et ça a donné erreur dans la résolution de K, puis on l'aidait à corriger la réponse, en lui apprenant à bien utiliser le bloc opérateur puis il a pu vérifier que sa réponse manuellement est incorrect et il l'a résolu de nouveau et finalement ça a donné une réponse correcte.

Lors de calcul avec les grands nombres, il nécessitait un peu plus de temps dans la résolution manuelle, alors que quand il l'a traité avec Scratch, juste un peu de temps lui a suffi,

Dans l'exercice du traçage de cercle et calcul du périmètre et aire, ça été un complexe pour lui de résoudre à la fois dans un même programme, et notons que ce niveau fait partis du niveau moyen, car l'utilisation de boucle est déjà évoquée. Alors il lui a fallu plus de temps et il a eu besoin qu'on le dirige un peu car la notion de la boucle, la demande à l'utilisateur (input () en python), l'utilisation de stylo pour le traçage (turtle en python) est tout nouveau pour lui.

Alors, on lui a conseillé de décomposer la tâche, c'est-à-dire essayer de tracer d'abord le cercle et voir comment ça marche, notons qu'il a essayé plusieurs angles pour avoir un cercle, il a fait 60° puis ça a donné un hexagone, alors il a changé au fur et à mesure la mesure de l'angle, il a remarqué que plus il augmente la mesure de l'angle, plus la figure ne tend pas vers une figure de cercle alors, il a opté de toujours diminuer de son premier (c'est-à-dire 60) la mesure de l'angle jusqu'à ce qu'il aura un cercle, et il s'est arrêté à l'angle de 10° ; puis demander à l'utilisateur le rayon pour pouvoir calculer le périmètre. Et il a amélioré son code, utilisé l'évènement 'quand le drapeau vert est cliqué' pour tracer d'abord le cercle, 'quand la touche espace est pressée' pour demander à l'utilisateur d'entrer la valeur du rayon puis 'quand ce Sprite est cliqué' pour calculer l'aire et le périmètre.

Dans la partie du niveau difficile, c'est-à-dire le jeu de devinette, on lui a expliqué l'objectif du jeu, que s'il y a un nombre aléatoire de 1 à 10 par exemple, l'utilisateur va deviner le nombre secret assortit du nombre aléatoire, et si la réponse de l'utilisateur est correcte, on va dire 'OK ! marina' mais sinon, on va toujours demander à l'utilisateur d'entrer un nombre jusqu'à ce qu'il donne la réponse correcte. Au début, il n'a pas trop compris, puis on l'a expliqué d'une autre manière, comme suit : si on a 10 boules numérotés de 1 à 10, une personne choisit une de ces boules, et lui aussi doit choisir un numéro de boule de 1 à 10, puis on compare son numéro à celui qui détient la boule et si son numéro de boule est égale au numéro de boule assortit alors il gagne, sinon il perd et doit recommencer à choisir un numéro puis le comparer et ainsi de suite, après ça il a compris le jeu et a fait le programme, ce qui lui a beaucoup plus dans ce jeu c'est qu'on a utilisé le bloc 'son' et on a enregistré sa voix en disant 'OK, marina' si la réponse de l'utilisateur est égale au nombre aléatoire assortit, 'diso, avereno indray' sinon .

On tient à mentionner aussi que le fait que les blocs de Scratch sont colorés, ça a beaucoup aidé Rayan à se souvenir le nom de chaque bloc et leur utilité.

D'après ces exercices, on a pu constater que dans le langage naturel, Rayan a déjà une pensée algorithmique, mais lorsqu'on lui a enseigner la pensée informatique à travers des exercices de math, des animations et de jeu en utilisant Scratch, ça a fait naitre la logique, l'algorithme plus avancé que celle que lui avait déjà, ça a augmenté sa créativité en voulant toujours améliorer son code et à essayer une nouvelle chose, le fait qu'il peut créer des animations et de jeux dans Scratch lui a beaucoup motivé de l'utiliser, or que s'il pratiquerait de plus en plus en création de jeu, sa compétence en algorithme, en logique, en créativité et en décomposition de tache va toujours augmenter .

Le fait que l'interface de Scratch est interactive, cela contribue à la motivation des élèves à programmer, dès que les élèves voient le lutin, (chat) par défaut, ils sont déjà contents et motivés à l'utiliser.

Maintenant on va voir et discuter les résultats de l'expérimentation avec Feno. Ce dernier possédait déjà aussi une pensée algorithmique, on l'a pu constater au moment de réalisation de calcul sur papier, il met des étapes de calcul pour résoudre les exercices du niveau facile. Mais il est un peu lent à la résolution de problème, il a eu besoin autant de temps que Rayan dans tous les exercices alors on n'a pas pu faire beaucoup de choses, et il a beaucoup eu d'accompagnement dans presque tous les exercices,, avec Rayan, on n'a pas donné des indices, il traite les exercices comme il les pensait et on lui a juste fait savoir qu'il peut améliorer son

programme, on lui donne l'idée et lui seul améliore son programme, ce qu'on veut en venir c'est que Feno ne possède pas beaucoup de motivation à la résolution de problème, et il manque aussi un peu de pensée logique. Prenons l'exercice niveau facile, calcul de A

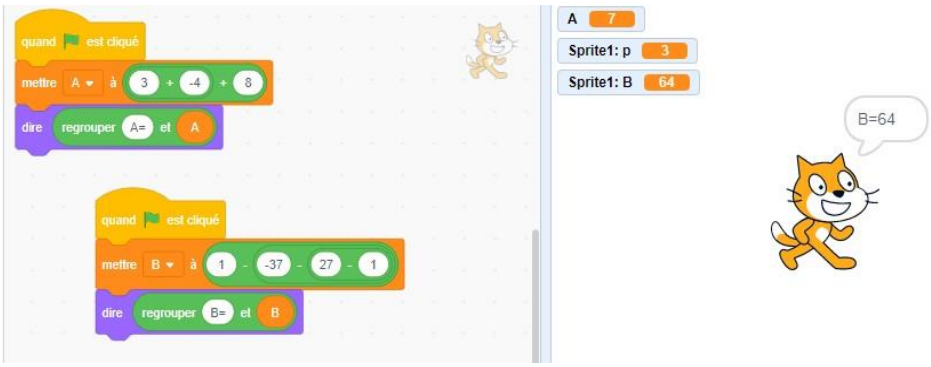
A= (+3) + (-4) + 8

On lui a expliqué comment calculer deux nombre en Scratch, on utilisateur le bloc opérateur, si on veut calculer la somme de deux nombres on utilise l'opérateur avec le signe plus (resp signe moins)

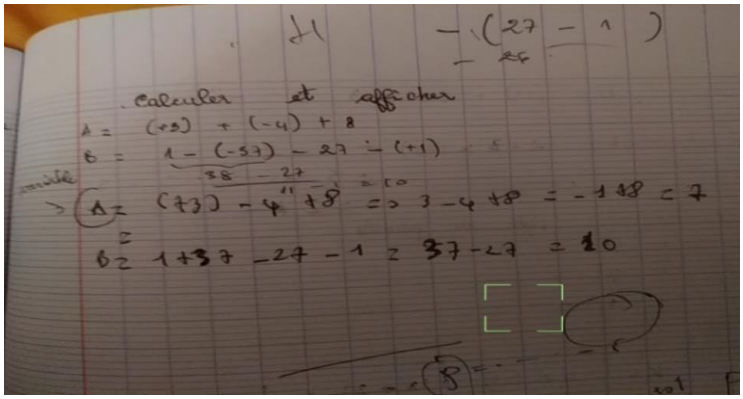
Pour calculer A, il a pris l'opérateur avec le signe plus, il a entre 3 et -4 ans le bloc, il a pensé quelque temps, on lui a dit maintenant on veut calculer trois nombres, alors qu'est ce qu'on fait ? il a répondu je n''ai pas d'idée', et c'est qu'entre la logique, pour calculer la somme de deux nombres, on a besoin une bloc d'opérateur plus, alors si on veut calculer la somme de trois nombres, il nous faut deux blocs.

Mais en lui faisant connaitre ça, dans ces autres exercices, on constate qu'il y ait de plus en plus de logique, par exemple en calculant B= 1 – (-37) – 27 – (+1) il utilisait trois blocs car il avait quatre nombre à calculer

Sur papier, il a trouvé la bonne réponse, mais la réponse sur Scratch a été incorrect



Résultat sur Scratch



Résultat sur papier

Alors on cherchait de trouver l'erreur, on constatait qu'il a rencontré la même erreur que Rayan dans l'utilisation de bloc opérateur, il apercevait que tous l'opérateur de B sont moins, alors il a utilisé des blocs d'opérateur moins, en décomposant son calcul on voit ce résultat



Il plaçait le premier bloc gauche dans le second bloc, dans la place de -37, ça forme un bloc puis il le plaçait dans le troisième bloc droit à la place de 27, on constate qu'il n'effaçait pas les nombres redondants (-37, 27), alors en exécutant son calcul ça donne différent nombres à chaque exécution, ici 64, autre exécution è26,333, donc on a trouvé ou était l'erreur

Puis recommençait son calcul, mais résultat était encore incorrect sur Scratch, ça a donné 12 ; pour comprendre l'erreur, on a encore décomposé son opération, le voici :



Il plaçait le premier bloc dans la place vide avant – du deuxième bloc et le troisième bloc dans la place vide du deuxième bloc après -. Ça a donné 12 car Scratch le calculait comme – (27-1) = -26 donc 38 – 26 ça a donné 12

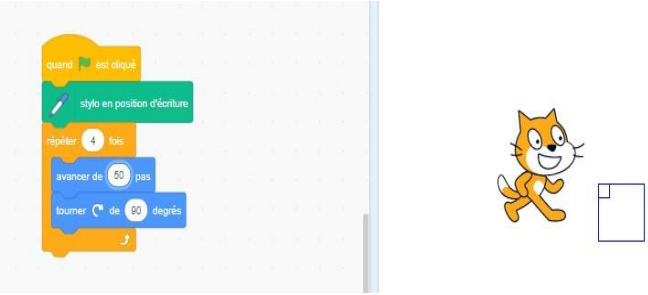
Donc il faut la corrigé, la réponse correcte est comme suivie :



Il faut introduire le premier bloc dans la place vide du deuxième bloc, ça forme un bloc et on introduit ce bloc dans le troisième bloc.

Il a connu aussi la même erreur que Rayan lors de calcul avec la multiplication, son problème a été la règle de priorité de calcul mais en le traite sur Scratch, il a pu comprendre que l'opérateur multiplication a un ordre de priorité supérieure que le plus ou le moins.

Lors du traçage du carré, il avait une réponse correcte mais il n'a pas utilisé le bloc « effacer tout » du stylo alors au début fait avancer de 4 pas, ça a donné un minuscule carré, puis il a changé en avancer 50 pas, le dernier minuscule carré figurait à l'intérieur du nouveau carré



Dans le niveau difficile, il n'a presque pas fait l'exercice, on lui beau a donné les indices mais il n'arrivait pas à faire l'algorithme du jeu, il a compris le concept mais il n'arrivait pas à faire le programme.

Alors on a rendu compte qu'il a besoin beaucoup de séance d'apprentissage de la pensée informatique, mais s'il continue à l'apprendre petit à petit, on est sûr qu'il va développer la pensée informatique, car d'après une durée d'une heure trente d'apprentissage, il a pu acquérir la compétence en logique, mais après quelques séances d'apprentissage, il pourrait développer des compétences en algorithmique plus complexe, de résolution de problèmes et en créativité. Sans oublier, il est motivé d'utiliser Scratch grâce à l'animation qu'offert Scratch, le fait que les élèves peuvent créer des animations leur motive beaucoup, et cela joue un rôle sur le

développement de sa pensée informatique, parce que plus un élève est motivé il peut tout apprendre.

3.3 Compétences développées chez les échantillons et limite de Scratch

3.3.1 Compétences développées :

Comme évoqué dans les sections précédentes, il y a un peu de différence sur la motivation des élèves scientifiques et littéraires constaté lors de l'expérimentation, Rayan est plus scientifique que Feno, et pendant l'expérimentation, on percevait deux attitudes différents chez eux, ils sont tous les deux motivés d'utiliser Scratch, mais sur l'effort de résolution de problème, Rayan est plus actif que Feno, ce dernier baisse les bras facilement s'il rencontre un problème complexe tandis que Rayan ne veut pas capituler devant un exercice difficile, lors de son exercice d'afficher à la fois les résultats de calcul de A,B,C,D, ce que l'on lui enseignait c'est d'afficher une variable avec son résultat, puis deux variables avec ses résultats, alors on lui a demandé comment alors afficher quatre variables et ses résultats en même temps, il réfléchissait un peu et dit « ahh ! Je sais, puis il a essayé son idée ».

Alors les compétences développées par les deux échantillons sont différentes et on pense que la situation abordée ci-dessus peut influencer le développement de la pensée informatique, mais si un élève littéraire est bien assisté dans les matières informatiques cela peut améliorer et développer sa pensée informatique, parce que toute personne la possède déjà même si elle ne le connait pas mais il faut la développer.

Alors, pour Rayan, ses compétences en logique, en algorithmique et en résolution de problème ont pu développer, Scratch a veillé sa créativité et sa motivation.

Pour Feno, ses compétences en logique et en algorithmique ont pu développer, et il a été aussi motivé d'utiliser le langage de programmation Scratch.

3.3.2 Limite de Scratch

Une des principales limites de cette étude réside dans l'insuffisance de l'échantillon, qui se compose uniquement sur deux échantillons, ce qui rend difficile l'évaluation du développement de la pensée informatique chez les élèves. De plus la courte durée de l'expérience, qui se limitait à une séance de 3heures peut ne pas suffire pour observer des progrès significatifs dans les compétences des élèves, rendant difficile l'évaluation précise de l'efficacité de l'enseignement de Scratch pour développer la pensée informatique.

Pour certains village, l'utilisation de l'outil numérique comme Scratch serait encore inaccessible, la plupart des écoles du village ne possède pas de laboratoire ou salle d'informatique. L'utilisation de Scratch comme outil d'apprentissage de la pensée informatique au secondaire à Madagascar est une belle initiative mais reste encore difficile, cela nécessite du temps et une bonne gestion de classe.

La manque de formation des enseignements sur l'apprentissage par outil numérique peut faire partie d'une limite de Scratch, les enseignants de collèges malgaches ne se familiarisent pas avec les outils numériques, donc ça rend difficile l'enseignement de la pensée informatique avec Scratch.

CONCLUSION

En conclusion, cette a permis de mettre en évidence l'impact positif de Scratch pour développement de la pensée informatique chez collégiens (6e selon l'expérimentation) à Madagascar. Les compétences développées par les deux élèves l'illustrent bien. Le premier, avec sa passion pour les sciences et sa débrouillardise, a su développer de ces compétences solides en algorithmique, en logique et en résolution de problème et en créativité. Sa motivation à explorer de nouvelles idées sur Scratch témoigne l'efficacité de cet outil pour stimuler l'engagement des élèves. De son côté, le deuxième, bien qu'initialement appréhensif, a progressivement gagné en confiance et en motivation grâce à l'interface conviviale de Scratch. Il a développé des compétences en logique et en algorithmique, même s'il a encore besoin d'assistance pour maitriser pleinement ces concepts ; cela souligne l'importance d'un accompagnement adapté pour maximiser l'apprentissage des élèves.

Cependant, cette étude a également révélé certaines limites, notamment en ce qui concerne l'accès aux ressources technologiques et nécessité de formation adéquate pour les enseignants. Pour que l'enseignement de la pensée informatique soit véritablement efficace à Madagascar, il est crucial d'investir dans des infrastructures et des formations qui permettront aux élèves de tirer pleinement parti des outils comme Scratch. Ainsi, cette recherche ouvre la voie à de futures études sur l'intégration de la pensée informatique dans le programme scolaire malgache, avec l'espoir que chaque élève puisse développer des compétences nécessaires pour naviguer dans un monde numérique en constante évolution.

Il est essentiel de donner des formations aux enseignants afin d'améliorer leur maitrise de la programmation (même une simple programmation comme Scratch) et de la pensée informatique, ainsi que les initiatives visant à améliorer l'accès aux technologies dans les écoles.

BIBLIOGRAPHIE

1. Wing, J.M. (2006). Computational Thinking. Communication of the ACM, 49(3), 33-35
2. Resnick, M., Maloney, J., Monroy-Hernandez, A., Rusk, N., Eastmond, E, Brennan, K., … & kafai, Y. (2009). Scratch : programming for all. Communication of the ACM, 52(11),60-67
3. Tchounikine, P. (2017). Initier les élèves à la pensée informatique et à la programmation avec Scratch. Laboratoire d'informatique de Grenoble
4. Sigayet, K., Tricot, A., & Blanc, N. (2021). Pensée informatique et activités de programmation, quels outils pour enseigner et évaluer ? in Atelier « Apprendre la pensée Informatique de la maternelle à l'Université », dans le cadre de la conférence Environnements Informatiques pour l'Aprrentissage Humain (EIAH) (pp. 68-75).
5. Sigayet, K. (2023). L'impact des outils utilisés pour l'apprentissage de la programmation et le développement de la pensée informatique au cycle 3(Doctoral dissertation, Université Paul Valéry-Montpellier III).
6. Bers,M.U. (2018) Coding and computational thinking in early childhhood : the impact of Scratch Jr in Europe, European Journal of STEM Education, 3(3),8.
7. Apedjinou, A., & Kouawo, C. A. A Formation des enfants du primaire à la programmation informatique avec Scratch : Observation et analyse du développement des compétences sous-jacentes à la pensée informatique. Revue Adjectif, T4.